

Práctica 5. Datos anomalos

Tratamiento de Datos, Grado en Ciencia de Datos - UV

Carlos Santiago Martinez Torres

2025-04-09

Contents

Introduccion y objetivos	1
Detección de inconsistencias en los datos	2
Ejercicio 1: Inspección Visual	2
Ejercicio 2: Valores especiales	3
Ejercicio 3: Valores perdidos y observaciones completas	3
Detección de outliers univariante	4
Ejercicio 4: Estadísticos básicos	4
Ejercicio 5: Métodos de detección de outliers	5
Ejercicio 6: Detección de outliers	6
Ejercicio 7: Comparativa de los métodos de detección de outliers	7
Imputación de datos	8
Ejercicio 8: Representación gráfica	8
Ejercicio 9: Imputación de datos perdidos con estadísticos básicos	9
Ejercicio 10: Representación gráfica	10

Introduccion y objetivos

El objetivo de esta práctica es comprobar si los conjuntos de datos con los que vamos a trabajar son consistentes y, por tanto, pueden ser empleados para realizar un análisis sin errores. Las observaciones contenidas en los datos pueden presentar diferentes tipos de inconsistencias, como valores perdidos (missing values), valores especiales y valores anómalos (outliers). Para evitar estas inconsistencias, estos datos deberán ser eliminados, corregidos o imputados. Siendo las etapas que pueden seguirse para obtener datos consistentes:

1. Detección de las inconsistencias.
2. Detectar el campo o campos que las generan y las dependencias entre ellos.
3. Corrección de errores.

Vamos a trabajar con el conjunto de datos `dirty_iris`, que ya conocemos, que contiene datos perdidos, valores especiales y anomalías de carácter numérico respecto a la distribución de los datos (`dirty_iris.csv`).

```
# Limpiamos el entorno.
rm(list=ls())

# Cargamos las librerías.
library(tidyr)
library(dplyr)
library(ggplot2)
library(readr)
```

Cargamos el conjunto de datos con el que vamos a trabajar.

```
dirty_iris <- read_csv("./data/dirty_iris.csv")
```

Detección de inconsistencias en los datos

Por lo general, no hay un convenio o una representación común para indicar un dato perdido o incluso determinados valores especiales. En los siguientes ejercicios vamos a analizar cómo detectar inconsistencias presentes en los datos.

Ejercicio 1: Inspección Visual

Inspecciona visualmente los valores del conjunto de datos (View) y ordena de mayor a menor y de menor a mayor cada variable para descubrir las diferentes inconsistencias que presenta el conjunto `dirty_iris`. Comenta los valores que te llamen la atención.

```
#view(dirty_iris)

summary(dirty_iris)

lista_col <- colnames(dirty_iris)
#"Sepal.Length" "Sepal.Width" "Petal.Length" "Petal.Width" "Species"

for (n in lista_col) {
  r <- dirty_iris %>%
    arrange(desc(.data[[n]]))
  print(r)
}

for (n in lista_col) {
  r <- dirty_iris %>%
    arrange(,.data[[n]]) # Por defecto es ascendentes
  print(r)
}
```

- Hay algunos datos identificados como `inf`.
- Cuando se ordena de menor a mayor, hay un valor de `Sepal Width` que es negativo.
- Ordenando, ya sea de mayor a menor o viceversa, hay alrededor de 2-3 valores anómalos.

Ejercicio 2: Valores especiales

Visualizar los datos puede ser muy revelador, pero en general, especialmente para conjuntos de datos grandes, realizaremos este análisis de forma automática. Los valores especiales más comunes en R son:

- NA (not available): missing value o dato perdido (is.na)
- NULL (empty set): elemento nulo de longitud 0 (is.null)
- NaN (not a number): indeterminación con resultado desconocido (is.nan)
- Inf (infinite): valor numérico de valor infinito (is.infinite)

Nota: A parte de estas funciones, is.numeric comprueba si la variable es numérica; is.finite comprueba la existencia de cualquier valor especial en un vector numérico pero devuelve FALSE para variables de texto. Reemplaza, en todas las columnas numéricas del data frame dirty_iris, los valores que sean especiales (usando is.finite) o negativos por NA. Nota: Puedes reemplazar los valores para cada variable del data frame usando mutate con la función ifelse.

```
sapply(dirty_iris, class) # Verificamos que si están las variables numéricas.
```

```
Sepal.Length Sepal.Width Petal.Length Petal.Width Species
"numeric"    "numeric"    "numeric"    "numeric"    "character"
```

```
# Si la columna es numerica, verificamos que el valor es inf o menor a 0 y lo cambiamos.
for (n in lista_col) {
  if (is.numeric(dirty_iris[[n]])) {
    dirty_iris[[n]][!is.finite(dirty_iris[[n]]) | dirty_iris[[n]] < 0] <- NA
  }
}
```

```
summary(dirty_iris) # Verificamos que los datos hayan sido cambiados
```

Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
Min. : 0.000	Min. : 0.000	Min. : 0.00	Min. : 0.100	Length:150
1st Qu.: 5.100	1st Qu.: 2.800	1st Qu.: 1.60	1st Qu.: 0.300	Class :character
Median : 5.750	Median : 3.000	Median : 4.50	Median : 1.300	Mode :character
Mean : 6.559	Mean : 3.439	Mean : 4.45	Mean : 1.582	
3rd Qu.: 6.400	3rd Qu.: 3.300	3rd Qu.: 5.10	3rd Qu.: 1.900	
Max. :73.000	Max. :30.000	Max. :63.00	Max. :23.000	
NA's :10	NA's :18	NA's :19	NA's :13	

Ejercicio 3: Valores perdidos y observaciones completas

• Calcula el número y porcentaje de observaciones completas (muestras sin ningún dato perdido) empleando complete.cases (detecta todos los registros sin datos faltantes).

```
# Todos los registros sin datos faltantes.
```

```
observaciones_completas <- complete.cases(dirty_iris)
```

```
# Número de observaciones completas
```

```
total_completos <- dim(dirty_iris[observaciones_completas, ])[1]
```

```
print(paste('Total observaciones completas :', total_completos))
```

```
[1] "Total observaciones completas : 94"
```

```
# Para el porcentaje, tomamos los casos favorables entre todos los posibles, y multiplicamos por 100.  
# Porcentaje de observaciones completas  
porcentaje <- round(total_completos / dim(dirty_iris)[1], 2) * 100  
  
print(paste('Porcentaje de observaciones completas : ', porcentaje, '%'))
```

```
[1] "Porcentaje de observaciones completas : 63 %"
```

- Emplea después la función `na.omit` para quedarnos solo con las filas con datos completos en un nuevo data frame `clean_iris`.

```
clean_iris <- dirty_iris %>%  
  na.omit()  
  
dim(clean_iris) # Comprobamos el nuevo df
```

```
[1] 94 5
```

```
sum(is.na(clean_iris)) # Verificamos que no hayan valores NA.
```

```
[1] 0
```

Detección de outliers univariante

Un outlier es una observación (o conjunto de observaciones) que no parecen consistentes con el resto de valores de esa variable. Por lo general, esas observaciones divergirán del patrón de comportamiento del resto de datos, es decir, presentarán una desviación respecto de la distribución de los datos. En esta sección, vamos a centrarnos en métodos de detección univariantes, así que, en los ejercicios, a modo de ejemplo, analizaremos solo la variable `Petal.Width`.

Ejercicio 4: Estadísticos básicos

La presencia de outliers puede producir una gran influencia en caracterizaciones sencillas de los datos, como la media y la desviación típica.

Empleado la función `summarise` y `across`, calcula la media, la desviación típica, mediana, la median absolute deviation from the median (MADM) y el interquartile range (IQR) de las variables numéricas. Hazlo para el data frame con observaciones completas `clean_iris` (que no tiene NA pero tiene menos observaciones) y para el data frame con datos faltantes `dirty_iris` (que presenta muchos NA pero tiene todas las observaciones).

- Ten en cuenta que no se puede realizar un análisis estadístico con datos faltantes, pero en la mayoría de los casos se puede optar por no considerar dichos valores al realizar los cálculos (parámetro `na.rm=TRUE`).
- Para obtener el valor MADM, escribe una función `madm(x)` que sea robusta a la presencia de valores NA. Recuerda que el MADM se calcula como $1.4826 \cdot \text{median}(|x - \text{median}|)$.
- Compara los resultados para los dos conjuntos en la variable `Petal.Width`.

```

# Funcion que recibe una columna y evalua la desviacion absoluta de la mediana 'median absolute deviation'
madm <- function(v) {
  mediana <- median(v, na.rm=TRUE) # Cada vez que 'opera el vector', considera los na.
  valor <- 1.4826 * median(abs(v - mediana), na.rm=TRUE)
  return(valor)
}

# Analisis para df sin NA
info_clean <- clean_iris %>%
  summarise(across(.col = where(is.numeric),
                    .fns = list (mean = mean, median = median, madm, IQR = IQR)))

# En este caso, se va a modificar la funcion base mean(), sd(), etc,
# ya que le pasamos un argumento extra que es na.rm...
# Por lo tanto, para aplicar la funcion, se debe llamar con ~

# Analisis para df con NA
info_dirty <- dirty_iris %>%
  summarise(across(.col = where(is.numeric),
                    .fns = list (
                      mean = ~ mean(., na.rm = TRUE),
                      median = ~ median(., na.rm = TRUE),
                      madm,
                      IQR = ~ IQR(., na.rm = TRUE))))

# Comparamos los resultados
resultados <- rbind(info_clean, info_dirty)
resultados

# A tibble: 2 x 16
  Sepal.Length_mean Sepal.Length_median Sepal.Length_3 Sepal.Length_IQR Sepal.Width_mean
                <dbl>                <dbl>          <dbl>          <dbl>                <dbl>
1                6.35                  5.8            1.04            1.3                3.30
2                6.56                  5.75            0.964          1.3                3.44
# i 11 more variables: Sepal.Width_median <dbl>, Sepal.Width_3 <dbl>, Sepal.Width_IQR <dbl>,
#   Petal.Length_mean <dbl>, Petal.Length_median <dbl>, Petal.Length_3 <dbl>,
#   Petal.Length_IQR <dbl>, Petal.Width_mean <dbl>, Petal.Width_median <dbl>, Petal.Width_3 <dbl>,
#   Petal.Width_IQR <dbl>

```

Ejercicio 5: Métodos de detección de outliers

Escribe una función que detecte los outliers de un vector x para cada uno de los siguientes métodos de detección de outliers univariantes (todas las funciones tienen que ser robustas a la presencia de valores NA).

- Regla 3 sigma `reglasigma(x)`: Considera que los datos siguen una distribución gaussiana. Datos típicos caracterizados por la media y la dispersión por la varianza. Valores por encima de 3 desviaciones típicas se consideran outliers.
- Identificador Hampel `reglahampel(x)`: Utiliza estimadores robustos y no presupone que los datos siguen una distribución gaussiana. Datos típicos caracterizados por la mediana y la dispersión por la median absolute deviation from the median (MADM). Valores por encima de 3 MADM se consideran outliers.
- Regla boxplot `reglaboxplot(x)`: A partir de los datos se calcula su diagrama de caja (boxplot) y se definen como datos típicos superiores hasta el cuartil 75%, x_s ; datos típicos inferiores hasta el cuartil 25%, x_i ; y la

dispersión se define como el rango intercuartílico ($IQR = xs - xi$). Valores por encima de $xs + 1.5 \cdot IQR$ o por debajo de $xi - 1.5 \cdot IQR$ se consideran outliers. Usa la función `quantile` para calcular los cuartiles.

- Percentiles `reglapercentil(x)`: Cualquier valor que esté fuera del rango del percentil 5% y 95% se puede considerar como atípico. Usa la función `quantile` para calcular los percentiles.

```
# Funcion para Regla 3 sigma
reglasigma <- function(v){
  media <- mean(v, na.rm=TRUE)
  sigma <- sd(v, na.rm=TRUE)
  return (v < ((media - (3*sigma)) | (media + (3*sigma))))
}

# Funcion para Identificador Hampel
reglahampel <- function(v) {
  mediana <- median(v, na.rm = TRUE)
  madm <- madm(v)
  return((v < (mediana - (3 * madm)) | (v > (mediana + (3 * madm)))))
}

# Funcion Regla boxplot
reglaboxplot <- function(v){
  xi <- quantile(v, probs = 0.25, na.rm=TRUE)
  xs <- quantile(v, probs = 0.75, na.rm=TRUE)
  iqr <- xs - xi
  return((v < (xi - (1.5 * iqr)) | (v > (xs + (1.5 * iqr)))))
}

# Funcion Percentiles
reglapercentil <- function(v) {
  xi <- quantile(v, probs = 0.05, na.rm=TRUE)
  xs <- quantile(v, probs = 0.95, na.rm=TRUE)
  return((v < xi) | (v > xs))
}
```

Ejercicio 6: Detección de outliers

Utiliza la función `mutate` y `across` para identificar los outliers en la variable `Petal.Width` con todos los métodos de detección de outliers implementados. Genera una columna nueva para cada método. Nota: podría extenderse la aplicación a todas las variables numéricas usando también `mutate`, `across` y `where`.

```
dirty_iris <- dirty_iris %>%
  mutate(across(.cols = Petal.Width,
    .fns = list ( reglasigma = reglasigma,
                  reglahampel = reglahampel,
                  reglaboxplot = reglaboxplot,
                  reglapercentil = reglapercentil)))

clean_iris <- clean_iris %>%
  mutate(across(.cols = Petal.Width,
    .fns = list ( reglasigma = reglasigma,
```

```
reglahampel = reglahampel,
reglaboxplot = reglaboxplot,
reglapercentil = reglapercentil)))
```

Ejercicio 7: Comparativa de los métodos de detección de outliers

En los métodos de detección de outliers analizados, examina los resultados comparando:

- Número de outliers detectados por método.

```
dirty_iris %>%
  select(contains('regla')) %>%
  summarise(across(everything(), sum, na.rm = TRUE))
```

```
# A tibble: 1 x 4
  Petal.Width_reglasigma Petal.Width_reglahampel Petal.Width_reglaboxplot Petal.Width_reglapercentil
      <int>                <int>                <int>                <int>
1         45                 3                 3                 11
```

```
clean_iris %>%
  select(contains('regla')) %>%
  summarise(across(everything(), sum, na.rm = TRUE))
```

```
# A tibble: 1 x 4
  Petal.Width_reglasigma Petal.Width_reglahampel Petal.Width_reglaboxplot Petal.Width_reglapercentil
      <int>                <int>                <int>                <int>
1         32                 3                 3                 9
```

- Los valores etiquetados como outliers (ordenados de menor a mayor).

```
dirty_iris %>%
  filter(TRUE) %>%
  arrange(Petal.Width) %>%
  head()
```

```
# A tibble: 6 x 9
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species Petal.Width_reglasigma
      <dbl>      <dbl>      <dbl>      <dbl> <chr>      <lgl>
1         4.9         3.6         NA         0.1 setosa  TRUE
2         4.9         3.1         1.5         0.1 setosa  TRUE
3         4.8         3          1.4         0.1 setosa  TRUE
4         4.3         3          1.1         0.1 setosa  TRUE
5         5.2         4.1         1.5         0.1 setosa  TRUE
6         5.3         NA          NA         0.2 setosa  TRUE
# i 3 more variables: Petal.Width_reglahampel <lgl>, Petal.Width_reglaboxplot <lgl>,
#   Petal.Width_reglapercentil <lgl>
```

```
clean_iris %>%
  filter(TRUE) %>%
  arrange(Petal.Width) %>%
  head()
```

```
# A tibble: 6 x 9
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species Petal.Width_reglasigma
      <dbl>      <dbl>      <dbl>      <dbl> <chr>      <lgl>
1         4.9         3.1         1.5         0.1 setosa    TRUE
2         4.8         3         1.4         0.1 setosa    TRUE
3         4.3         3         1.1         0.1 setosa    TRUE
4         5.2         4.1         1.5         0.1 setosa    TRUE
5         4.8         3.1         1.6         0.2 setosa    TRUE
6         4.7         3.2         1.3         0.2 setosa    TRUE
# i 3 more variables: Petal.Width_reglahampel <lgl>, Petal.Width_reglaboxplot <lgl>,
#   Petal.Width_reglapercentil <lgl>
```

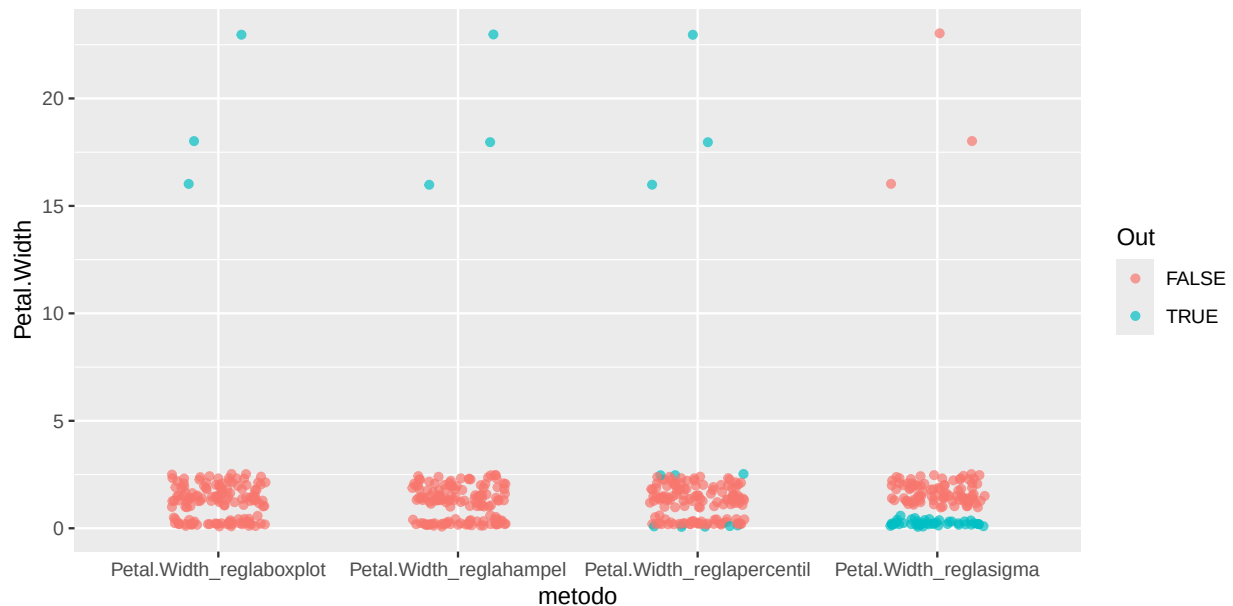
Imputación de datos

Ejercicio 8: Representación gráfica

Representa gráficamente los datos de Petal.Width (distribución de puntos en el eje y) para todas las reglas de detección implementadas (repartidas en el eje x) mostrando los valores identificados como típicos y los identificados como outliers en diferente color. Usa la función `pivot_longer` para poder agrupar por reglas y colorear los outliers fácilmente.

Nota: usa en `geom_point` con la opción `position='jitter'` para mejorar la legibilidad del gráfico.

```
dirty_iris %>%
  pivot_longer(cols = contains('regla'),
               names_to = "metodo",
               values_to = "Out") %>%
  filter(!is.na(Petal.Width)) %>%
  ggplot(aes(x = metodo, y = Petal.Width, colour = Out)) +
  geom_point(position = position_jitter(width = 0.2), alpha = 0.7)
```



```
clean_iris %>%
  pivot_longer(cols = contains('regla'),
               names_to = "metodo",
               values_to = "Out") %>%
  filter(!is.na(Petal.Width)) %>%
  ggplot(aes(x = metodo, y = Petal.Width, colour = Out)) +
  geom_point(position = position_jitter(width = 0.2), alpha = 0.7)
```



Ejercicio 9: Imputación de datos perdidos con estadísticos básicos

La opción más sencilla es imputar con un valor estimado de los valores conocidos (por ejemplo, con la media, la mediana, o la moda). Se puede realizar una imputación generalizada (usando todos los valores de la variable) o una imputación a partir de casos similares (la estimación se realiza teniendo en cuenta otra variable):

- Imputa los missing data en Petal.Width empleando la media. Nota: Puedes reemplazar los valores vacíos usando mutate con ifelse creando una nueva variable Petal.Width1.

```
dirty_iris <- dirty_iris %>%
  mutate(Petal.Width1 = ifelse(is.na(Petal.Width),
                              mean(Petal.Width, na.rm = TRUE),
                              Petal.Width))
```

- Imputa los missing data en Petal.Width empleando en esta ocasión la media para cada tipo de Species (usar group_by y mutate) creando una nueva variable Petal.Width2.

```
dirty_iris <- dirty_iris %>%
  group_by(Species) %>%
  mutate(Petal.Width2 = ifelse(is.na(Petal.Width),
                              mean(Petal.Width, na.rm = TRUE),
                              Petal.Width))
```

Ejercicio 10: Representación gráfica

Vamos a comparar ahora los dos métodos de imputación gráficamente eliminando los outliers para una mejor visualización.

- Recalcula las reglas de identificación de outliers (Ejercicio 6) para la nueva variable Petal.Width2.
- Representa los dos tipos de imputación empleados en un gráfico de dispersión (Petal.Width1 vs. Petal.Width2) eliminando los outliers identificados con la regla reglapercentil y poniendo en diferente color los datos originales y los imputados.

```
dirty_iris <- dirty_iris %>%  
  mutate(outlier_percentil = reglapercentil(Petal.Width2)) %>%  
  mutate(tipo_dato = ifelse(is.na(Petal.Width), "Imputado", "Original"))  
  
dirty_iris %>%  
  filter(outlier_percentil == FALSE) %>%  
  ggplot(aes(x = Petal.Width1, y = Petal.Width2, color = tipo_dato)) +  
  geom_point(size = 3, alpha = 0.7)
```

